



UNIVERSITÀ
DI TRENTO

Let's talk about JavaScript Objects

1

Javascript objects

JavaScript can be confusing for Java / C++ developers as it's all dynamic, all runtime, and it **has no classes at all**

It's all just instances (objects)

Even the "classes" we will simulate (introduced in ES5) are just a function object

Javascript objects as data structures

Javascript objects are collections of named values (properties)

```
var person = {  
  firstName: "Giovanna",  
  lastName: "Varni",  
  birthyear: 1900};
```

...and possibly methods

```
var person = {firstName: "Giovanna",  
  lastName: "Varni", birthyear:1900,  
  fullName: function() {return  
    this.firstName + " " + this.lastName;}};
```

Accessing properties

`person.firstName`

is equivalent to

```
person["firstName"];
```

and to

```
var x="firstName";  
person[x];
```

Dynamic management of objects

- You can dynamically add new properties to an existing object by simply giving it a value:

```
var person = {firstName: "Giovanna",
              lastName: "Varni",
              birthyear:1900};

person.birthplace="Genova";
```

- You can also delete existing properties:

```
delete person.firstName;
```

Other ways to create objects

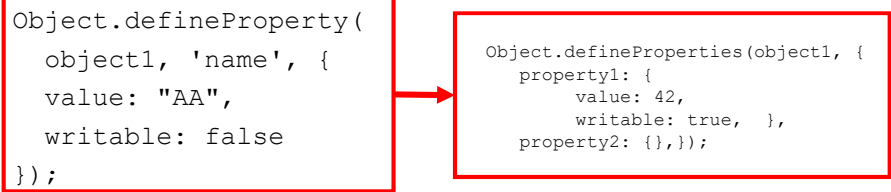
- Javascript objects can also be created empty and populated later

```
var person = {};

person.firstName="Giovanna";
person.lastName="Varni";
person.birthyear=1900;
person.fullName=function() {
    return this.firstName + " " +
    this.lastName;};
```

Other ways to create object

```
var object1 = new Object();
Object.defineProperty(
  object1, 'name', {
    value: "AA",
    writable: false
  });
object1.name = 77;
document.write(object1.name);
```



```
Object.defineProperties(object1, {
  property1: {
    value: 42,
    writable: true,
  },
  property2: {},});
```

OUTPUT:
AA

(but no error)

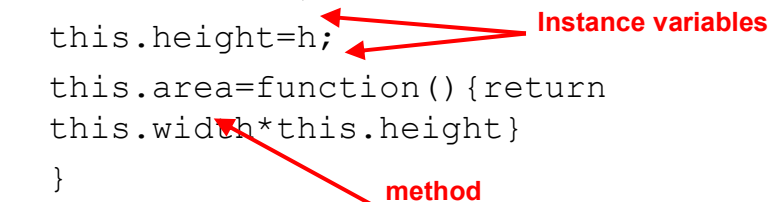
if writable: true:

OUTPUT:
77

Object constructors

Object constructors: templates for creating objects of a certain type (somehow similar to the concept of "class")

```
function Rectangle(w, h) {
  this.width=w;
  this.height=h;
  this.area=function(){return
  this.width*this.height}
}
a=new Rectangle(3,4);  a.area() => 12
a.width => 3
```



The constructor function is JavaScript's version of a class

Enumerating objects' properties

- We can use the for...in loop for looping through the properties of an object:

```
for (key in object) {
    // code block to be executed }
```

- Example:

```
x = new Rectangle(3,4);
for (let e in x) {
    console.log(e+" "+x[e]);}
```

Another example

```
<!DOCTYPE html>
<html>
<body>

<p id="demo"></p>

<script>
var person = {firstName:"Giovanna",
               lastName:"Varni",
               birthyear:1900};
person.birthplace="Genova";

let txt = "";
for (let x in person) {
    txt += person[x] + " ";
}

document.getElementById("demo").innerHTML = txt;
</script>

</body>
</html>
```

Something weird here...

```
<script>
p = (n,s) => document.write(n+": "+s+"<br>");

function Rectangle(w, h) {
  this.width=w;
  this.height=h;
  this.area=function() {
    return this.width*this.height}}

a=new Rectangle(2,3);
b=new Rectangle(2,3);
p(a.area(),b.area());
a.area=function(){return this.width*5}
p(a.area(),b.area());
</script>
```

OUTPUT:

6:6
10:6

We need to find a solution

- The approach we have shown is not the most efficient in terms of memory allocation, as for every Rectangle we instantiate the area method!
- Using prototype will help us!

```
Rectangle.prototype.area = function() {
  return this.w*this.h }
```

- The Prototype is a **separate naming space**, shared by all instances of the same constructor/class

Prototype

```
function Person(first, last) {  
    this.firstname = first;  
    this.lastname = last;}  
  
Person.prototype.fullname= function () {  
    return this.firstname+" "+this.lastname};  
  
Person.prototype.nickname="The Best";  
  
let person1 = new Person('Dorothea', 'Wierer');  
let person2 = new Person('Valentino', 'Rossi');  
console.log(person1.nickname);  
console.log(person2.nickname);  
Person.prototype.nickname="The Champion";  
console.log(person1.nickname);  
console.log(person2.nickname);
```

OUTPUT:
The Best
The Best
The Champion
The Champion

**Are there predefined objects in
JavaScript?**

Predefined objects

JavaScript has a number of predefined objects, e.g.:

```
var x1 = new Object();  
var x2 = new String();    // do not use it!  
var x3 = new Number();    // do not use it!  
var x4 = new Boolean();    // do not use it!  
var x5 = new Array();      // do not use it! Use []  
var x6 = new RegExp();     // do not use it! Use /( )/  
var x7 = new Function();   // do not use it!  
var x8 = new Date();
```

The native prototypes should **never** be extended unless it is for the sake of compatibility with newer JavaScript features.

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects

Arrays

- Arrays are a special type of objects
- ...special:

In JavaScript arrays use numbered indexes whereas objects use named indexes

- Arrays can be inhomogeneous

```
const person = ["Giovanna", "Varni", 1900];
```

Arrays

- Arrays can be declared in two ways:

```
a=[];
a = new Array(); //discouraged
```

- Operating on arrays:
 - Assignment and accessing elements

```
const cars = ["Fiat", "Opel", "BMW"];
let car = cars[0];
```

```
let text = "";
for (let x of cars) {
  text += x;}
}
```

We can do that
because arrays are
iterable

Arrays methods

Basic Array Methods

Array.length
Array.toString()
Array.at()
Array.join()
Array.pop()
Array.push()

Alpabetic Sort

Array.sort()
Array.reverse()
Array.toSorted()
Array.toReversed()
Sorting Objects

source: w3schools

Array.shift()
Array.unshift()
Array.delete()
Array.concat()
Array.copyWithin()
Array.flat()
Array.splice()
Array.toSpliced()
Array.slice()

Numeric Sort

Numeric Sort
Random Sort
Math.min()
Math.max()
Home made Min()
Home made Max()

Array Find and Search Methods

Array.indexOf()
Array.lastIndexOf()
Array.includes()

Array.find()
Array.findIndex()
Array.findLast()
Array.findLastIndex()



obj1+obj2

19

How + works on objects?

- In JavaScript, you cannot directly add objects together
- A two step procedure:
 - Converting to a primitive datatype:
 - It tries to convert the object using, in this order:
 - **Symbol.toPrimitive()** (if it exists)
 - **valueOf()** (inherited by Object)
 - **toString()** (inherited by Object)
 - Performing the operation:
 - Concatenation
 - Sum

So, there are no classes in JavaScript?

21

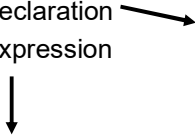
Classes: where they come from...

- JavaScript classes are primarily syntactic sugar over JavaScript's existing prototype-based inheritance
- The class syntax *does not* introduce a new object-oriented inheritance model to JavaScript

Classes

- JavaScript classes were introduced in ES6 (2015)
- A class can be defined in two ways:

- Class declaration
- Class expression



```

const Rectangle = class {
  constructor(height, width){
    this.height = height;
    this.width = width; }
  dimensions() {
    return this.height+" "+this.width;}}

class Rectangle {
  constructor(height, width){
    this.height = height;
    this.width = width; }
  dimensions() {
    return this.height+" "+this.width;}}
  
```

Unlike function declarations, classes declaration are hoisted, but they are not initialized until their declaration is reached, so they behave like let and const

Classes

- A JavaScript class is a kind of function

```

class Rectangle {
  constructor(height, width){
    this.height = height;
    this.width = width; } }

document.write(typeof Rectangle);
  
```

function

Classes variables

- There are two types of variables in JavaScript classes:
 - **instance variables**
 - **class variables**

```
class Rectangle {
  constructor(height, width){
    this.height = height;
    this.width = width; } }
```

- Instance variables belong to each instance of the class.
- They are defined inside the constructor function using "this"
- Each instance of the class has its own copy of the instance variables.
- They can be accessed / modified using the dot or the bracket notation

Classes variables

- There are two types of variables in JavaScript classes:
 - **instance variables**
 - **class variables**

```
class Rectangle {
  static species = "geometric";
  constructor(height, width){
    this.height = height;
    this.width = width; } }
```

- Class variables belong to the class itself, not to any instance of the class
- They are defined outside the constructor function using static
- There is only one copy of the class variable for the whole class
- It can be accessed and modified using the class name and the dot or the bracket notation

Classes fields

- ES2022's class fields allow you to declare and define instance properties / methods outside the constructor for improved readability

```
class Rectangle {
  static species = "geometric";
  #height;
  #width;

  constructor(height, width){
    this.#height = height;
    this.#width = width }
}
```

Classes inheritance

```
class Person {
  constructor(first, last ) {
    this.firstname = first;
    this.lastname = last;
  }
  fullname() {
    return this.firstname+" "+this.lastname;
  } }

```

```
class Athlete extends Person {
  constructor(first, last, sport ) {
    super(first,last);
    this.sport=sport;
  }
}
```

```
let person1 = new Athlete('Dorothea', 'Wierer', 'Biathlon');
let person2 = new Athlete('Valentino', 'Rossi', 'Moto GP');

console.log(person1.fullname()+" "+person1.sport);
console.log(person2.fullname()+" "+person2.sport);
```

Let's quiz!

- Quale sarà l'output della seguente riga di codice?

```
console.log(typeof typeof 1)
```

- a. string
- b. number
- c. NaN
- d. true

29

Let's quiz!

- Determinare l'output del seguente frammento di codice:

```
<!DOCTYPE html>
<html>
<body>
<script>
var test = function() {
  console.log('1');
}

test();

function test() {
  console.log('2');
}

test();

</script>
</body>
</html>
```

30

Let's quiz!

- Determinare l'output di ogni chiamata di p1 e p2 eseguita dal seguente frammento di codice:

```
p1=(n)=>document.write(n+"<br>");
p2=(n,s)=>document.write(n+": "+s+"<br>");

function f(){
  let b="0"
  for (a=0; a<5; a++)
    b=b+a;
  p1(b);
}

try{
  f();
  p1(a);

  p2(a*null,b);

}

catch (err) {p2("ERROR", err.message);}
```

31